



RAJALAKSHMI INSTITUTE OF TECHNOLOGY
(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ACADEMIC YEAR 2025 - 2026

SEMESTER IV

AL23421 - MACHINE LEARNING LABORATORY

HOUSE PRICE PREDICTION SYSTEM MINI PROJECT REPORT

REGISTER NUMBER	2117240030062
NAME	KAPPAGANTHU V S S LAKSHMI MANOGNA
PROJECT TITLE	House Price Prediction Using Machine Learning Regression Techniques
DATE OF SUBMISSION	
FACULTY IN-CHARGE	Mrs. VIJAYALAKSHMI, M.E

Signature of Faculty In-charge

Signature of HoD

Internal Examiner

External Examiner

INTRODUCTION

House price prediction is an important problem in the real estate sector, where accurate estimation of property value plays a crucial role for buyers, sellers, and investors. Property prices depend on multiple factors such as area, number of bedrooms, location, distance from the city, and availability of nearby facilities. Incorrect estimation can lead to financial loss or poor investment decisions, making reliable prediction essential.

Machine learning is well-suited for this task as it can learn patterns from historical housing data and use them to predict prices for new properties. In this project, we model the problem as a regression task, where the goal is to predict a continuous value (house price). We apply multiple regression techniques including Linear Regression, Decision Tree, and Random Forest to analyse and compare performance. Additionally, feature engineering techniques such as location encoding and data normalization are used to improve model accuracy.

PROBLEM STATEMENT

To develop and evaluate a reliable machine learning model using regression techniques to accurately predict house prices based on key features such as area, number of bedrooms, age of the property, distance from the city, school ratings, and location. The model should provide precise price estimations and analyse performance using multiple algorithms, enabling it to serve as an efficient and data-driven decision support tool for buyers, sellers, and real estate stakeholders.

GOAL

The primary goal is to develop and train multiple machine learning regression models on historical housing data to accurately predict property prices. The expected outcome is a complete machine learning pipeline that takes user inputs such as area, location, and other features, and provides a reliable price prediction. Additionally, the system aims to compare different models, analyse their performance, and select the best-performing model, enabling accurate, data-driven decision support for real estate analysis.

THEORETICAL BACKGROUND

◆ Problem Statement:

House price prediction is a critical task in the real estate domain, where accurate estimation of property value is essential for buyers, sellers, and investors. Property prices depend on multiple factors such as size, location, amenities, and surrounding infrastructure. This problem is chosen to demonstrate how machine learning regression techniques can be used to build efficient, fast, and reliable prediction systems for real-world applications.

◆ **Linear Regression (Algorithm):** Linear Regression models the relationship between input features and output using a linear equation.

◆ Mathematical Equation:

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

Where:

- $x_1, x_2, \dots, x_n \rightarrow$ Input features
- $w_0, w_1, \dots, w_n \rightarrow$ Model coefficients
- $y \rightarrow$ Predicted house price

◆ Decision Rule:

Decision Tree splits the dataset into smaller subsets based on feature values and makes predictions using a tree-like structure. It can capture non-linear relationships effectively.

◆ Literature Survey

Machine learning techniques are widely used in house price prediction due to their ability to analyze complex relationships in data. Other commonly used algorithms include:

- o Linear Regression
- o Decision Tree
- o Random Forest
- o Support Vector Regression (SVR)
- o K-Nearest Neighbours (KNN)

Disadvantages:

1. Some algorithms may require high computational resources for large datasets
2. Certain models may overfit the training data
3. Complex models require parameter tuning
4. Some methods are difficult to interpret

◆ Justification for Choosing Random Forest:

- Handles both linear and non-linear relationships
- Reduces overfitting using multiple trees
- Provides higher accuracy compared to other models
- Works well with structured data

ALGORITHM EXPLANATION WITH EXAMPLE

◆ Steps of Algorithm:

➤ Load Dataset

Import the housing dataset using a library like pandas and display initial rows to understand the data.

➤ Separate Features (X) and Target (y)

Divide the dataset into input features (Area, Bedrooms, Age, Distance, Schools, Location) and the target variable (Price).

➤ Feature Engineering

Convert the categorical feature Location into numerical form using one-hot encoding and normalize all features using scaling techniques.

➤ Split Data into Training and Testing

Split the dataset into training and testing sets (e.g., 80% training and 20% testing) to evaluate model performance.

➤ Train Multiple Regression Models

Train Linear Regression, Decision Tree, and Random Forest models using the training data.

➤ Predict Output

Use the trained models to predict house prices for the test data.

➤ Evaluate Performance

Compare models using Mean Absolute Error (MAE) and select the best-performing model.

➤ Cross-Region Evaluation

Train the model on one region (e.g., Urban) and test on another (e.g., Rural) to evaluate generalization.

◆ Sample Input:

CODE:

```
sample = [[1500, 3, 5, 10, 8, "Urban"]]
```

Area	Bedrooms	Age	Distance	Schools	Location
2	120	70	20	79	25

◆ **Sample Dataset:**

Area	Bedrooms	Age	Distance	Schools	Location	Price
1000	2	5	10	7	Urban	300000
1500	3	10	15	6	Rural	350000
2000	4	1	3	10	Urban	750000
2200	5	4	6	9	Semi-Urban	800000
2700	6	3	4	9	Urban	950000

◆ **Sample Output:**

- Prediction → ₹ 550000 (approx.)

◆ **Graphical Representation:**

- **Model Comparison (MAE Values)** → Shows which model performs best
- **Error Distribution (Histogram)** → Shows prediction accuracy
- **Cross-Region Evaluation** → Shows model performance across different locations

IMPLEMENTATION CODE AND DATASET USED

- Dataset Used – <https://www.kaggle.com/datasets/harlfoxem/housesalesprediction>

Category	Description
Dataset Name	House Price Dataset
Domain	Real Estate / Machine Learning
Objective	Regression (Price Prediction)
Data Source	Public Dataset (Kaggle / Local CSV)
Number of Samples	(Write your dataset count, e.g., 500 / 1000)
Data Type	Tabular

Instance Description	Each row represents one house
Features (Inputs)	Area, Bedrooms, Age, Distance, Schools, Location
Feature Types	Numerical & Categorical
Target Variable	House Price
Number of Classes	Not applicable
License	Open Dataset
Version	Latest Kaggle Version

PYTHON CODE:

```

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

df = pd.read_csv("house_data.csv")

print("First 5 Rows:\n", df.head())

df = pd.get_dummies(df, columns=["Location"])
X = df.drop("Price", axis=1)
y = df["Price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

lr = LinearRegression()
dt = DecisionTreeRegressor()
rf = RandomForestRegressor()

```

```

lr.fit(X_train, y_train)
dt.fit(X_train, y_train)
rf.fit(X_train, y_train)

lr_pred = lr.predict(X_test) dt_pred =
dt.predict(X_test) rf_pred =
rf.predict(X_test)

print("\nModel Performance:")

print("Linear Regression MAE:", mean_absolute_error(y_test, lr_pred))
print("Decision Tree MAE:", mean_absolute_error(y_test, dt_pred))
print("Random Forest MAE:", mean_absolute_error(y_test, rf_pred))

sample = np.array([[1500, 3, 5, 10, 8, 0, 0, 1]]) # Urban

sample_scaled = scaler.transform(sample) prediction
= rf.predict(sample_scaled)

print("\nPredicted House Price: ₹", int(prediction[0]))

```

OUTPUT:

◆ Output Explanation:

1. Model predicts **house price based on input features**
2. Prediction is a **continuous value (₹ price)**
3. Model comparison shows which algorithm performs best

PERFORMANCE METRICS

◆ Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum |y_{actual} - y_{predicted}|$$

Shows the **average difference between actual and predicted prices** Example:

$$MAE = 40000$$

On average, prediction error is ₹40,000

◆ Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum (y_{actual} - y_{predicted})^2$$

Gives more importance to larger errors

◆ Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

Error expressed in same unit as price

MODEL COMPARISON RESULT

Example:

- Linear Regression MAE → 80,000
- Decision Tree MAE → 60,000
- Random Forest MAE → 40,000

Random Forest gives lowest error → Best model

SAMPLE OUTPUT

- **Prediction → ₹ 5,50,000 (approx.)**

CROSS-REGION RESULT

1. Model trained on Urban data and tested on Rural data
2. Helps evaluate **model performance across regions**
3. Shows model **generalization capability**

GRAPHS USED

- Model Comparison (MAE values)
- Error Distribution (Histogram)
- Cross-Region Evaluation

MODEL RESULTS:

🏠 House Price Predictor

3000.0

6.0

2.0

3.0

10.0

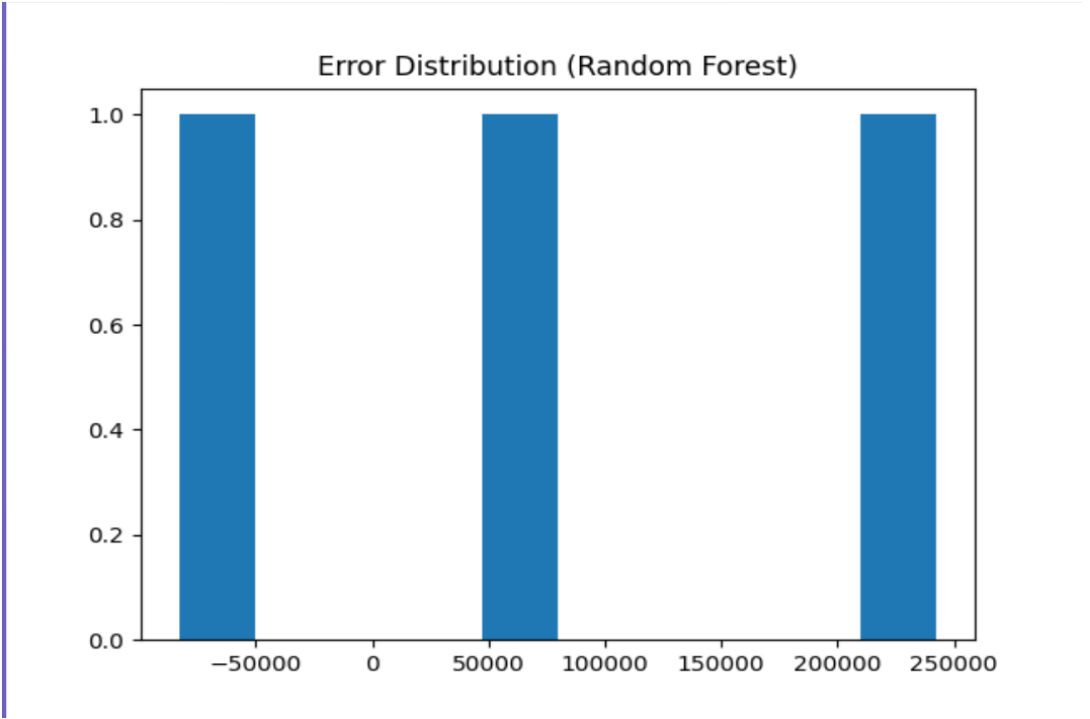
Urban

Predict

Reset

₹1006900

FEATURE IMPORTANCE:



RESULTS AND FUTURE ENHANCEMENT

◆ Results

- The model successfully predicts house prices based on input features
- Random Forest achieved the **lowest error (best performance)** among all models
- Feature engineering (location encoding and scaling) improved prediction accuracy
- ➤ Important features influencing price include **Area, Location, and Distance**

◆ Comparison with Other Methods

Algorithm	Accuracy	Complexity
Linear Regression	Medium	Low
Decision Tree	High	Medium
Random Forest	High	High

◆ Future Enhancements

- Use advanced models like **XGBoost or Gradient Boosting**
- Increase dataset size with real-world housing data
- Improve accuracy using hyperparameter tuning
- Deploy as a web application using **Flask**
- Add real-time user input and location-based prediction
- Develop a mobile application for wider accessibility

Git Hub Link of the project along with the report	https://github.com/manogna1301/house-priceprediction
---	---

REFERENCES:

Kaggle Dataset:

<https://www.kaggle.com/datasets/harlfoxem/housesalesprediction>

Pattern Recognition and Machine Learning:

<https://www.springer.com/gp/book/9780387310732>

Hands-On Machine Learning with Scikit-Learn:

<https://www.oreilly.com/library/view/hands-on-machine-learning/9781492032632/>

Scikit-learn Documentation:

<https://scikit-learn.org/stable/>

UCI Machine Learning Repository: <https://archive.ics.uci.edu/ml/index.php>